

One Man's Trash is Another Man's Treasure

One sentence description – agentic program that parses, understands, and finishes old, abandoned project repos on GitHub

Past project reference – new project! <https://github.com/Starnaphie/archaeologist>

Planned technologies

- Frontend: React
- Backend: Python with [FastAPI](#)

The agentic workflow will use the [OpenAI API Platform](#) along with LangGraph or LangChain for orchestration, while repository analysis will use GitPython, Tree-sitter, and vector search tools such as FAISS or Chroma. Mermaid.js or Graphviz may be used to generate repository architecture diagrams.

First deliverable

The first deliverable will allow a user to submit a GitHub repository URL and receive an automatically generated “archaeology report” describing the repository’s likely purpose, important files, architecture, and incomplete or abandoned features. This workflow forces us to integrate repository ingestion, code parsing, retrieval, agent reasoning, and frontend visualization into a single end-to-end system. The initial version will focus on small-to-medium Python repositories.

Rough architecture for first deliverable

The system will consist of several major components:

1. a repository ingestion component that clones repositories and retrieves commit history
2. a repository indexing component that chunks and embeds source files for retrieval

3. a static analysis component that extracts dependency graphs and module relationships
4. an agent reasoning component that infers repository purpose and summarizes functionality
5. a visualization component that generates architecture diagrams
6. a report generation component that produces the final archaeology report webpage

The main data flowing through the system includes repository metadata, code/document chunks, embeddings, dependency graphs, commit history summaries, and generated repository explanations. The system will store indexed repository information in a vector database to support retrieval and multi-step agent reasoning.

After first deliverable goals

- Multi-agent reasoning system with specialized “historian,” “architect,” and “bug prediction” agents
- Automatic README and documentation generation
- Detection of abandoned or partially implemented features
- Interactive repository Q&A (“What does this file do?”, “What feature was never finished?”)
- Timeline visualization showing architectural evolution across commits
- Support for additional programming languages beyond Python
- Improved architecture diagrams and dependency visualizations
- Evaluation on abandoned student projects and poorly documented open-source repositories